Deliverable 1: Requirements, Architecture and Project Setup

CS455: Introduction to Software Engineering · Instructor: Prof. Priyanka Bagade

Concord: A Collaborative Code Execution and Debugging Platform

Pallav Rastogi (230731) · Aamir Ahmad (230010) · Aryamann Srivastava (230211) · Tattwa Shivani (231089) · Sanchit Arora (230907)

This is the Deliverable 1 document for Concord. Deliverable 0 proposed the platform: its user roles, the code-execution workflow, the contended resources, the AI assistant and the technology stack. Building on that approved proposal, this submission contains the requirements specification, the use-case model and user stories, the functional and non-functional requirements, the initial requirements traceability matrix, the architecture and design document with component and sequence diagrams, the design-pattern mapping, the architecture decision records, the Jira project with its initial backlog, the Jira and GitHub integration, the initial sprint plan, and the AI engineering log for the requirements and architecture activities.

Contents

1 Requirements Specification **2**
- 1.1 Stakeholders . . . 2
- 1.2 User Requirements (Natural Language) . . . 2
- 1.3 Structured Specifications . . . 2

2 Use-Case Diagram and User Stories **4**
- 2.1 Actor Taxonomy . . . 4
- 2.2 UML Use-Case Diagram . . . 4
- 2.3 User Stories . . . 4

3 Functional and Non-Functional Requirements **5**
- 3.1 Functional Requirements . . . 5
- 3.2 Non-Functional Requirements . . . 7
- 3.3 Metrics for Non-Functional Requirements . . . 8
- 3.4 Requirements Validation . . . 8

4 Initial Requirements Traceability Matrix **8**
- 4.1 User Requirement to Functional Requirement, Use Case and Specification . . . 8
- 4.2 Critical Requirements to Jira Issue and Planned Test . . . 9
- 4.3 Functional Subsystem to Governing Non-Functional Requirements . . . 9

5 Architecture and Design Document **10**
- 5.1 System Context . . . 10
- 5.2 Container Diagram (System Architecture) . . . 11
- 5.3 Components . . . 12
- 5.4 Finite Resource and Calculation Rule . . . 12

6 Component and Sequence Diagrams **13**
- 6.1 Component Diagram . . . 13
- 6.2 SD-1: Real-Time CRDT Edit Flow . . . 13
- 6.3 SD-2: Priority-Ordered Execution Slot Allocation . . . 14
- 6.4 SD-3: Human-Gated AI Assistant Flow . . . 14
- 6.5 SD-4: Sandboxed Execution and Result Broadcast . . . 15

7 Design-Pattern Mapping **15**
- 7.1 General Design Patterns . . . 15
- 7.2 AI-Specific Architectural Patterns . . . 15

8 Architecture Alternatives and Rejected Decisions (ADRs) **16**

9 Jira Project with Initial Backlog **17**
- 9.1 Epics . . . 17
- 9.2 Backlog (excerpt; the full backlog of 37 issues is maintained in Jira) . . . 17

10 Jira and GitHub Integration **17**

11 Initial Sprint Plan **18**

12 AI Engineering Log **19**
- 12.1 Method . . . 19

1 Requirements Specification

1.1 Stakeholders

- **Students (primary users):** require a shared coding environment for pair programming, laboratory assignments and live demonstrations.
- **Instructors and teaching assistants:** review student code and running programs without taking control of the editor.
- **Course staff and operators:** deploy the platform, monitor system health, run load tests and keep resource usage within free-tier cloud limits.

1.2 User Requirements (Natural Language)

- **UR-01 (Account Registration and Authentication):** Users shall register with an email address, username and password, and shall log in to obtain an authenticated session.
- **UR-02 (Workspace and File Organization):** Users shall create projects, create nested files and folders, rename, move or delete them, and navigate a file tree.
- **UR-03 (Real-Time Multi-User Editing):** Project members shall edit the same file simultaneously, with changes propagated to all participants and with each collaborator's cursor visible.
- **UR-04 (Multi-Language Sandboxed Execution):** Users shall run Python, JavaScript or C++ programs with a single action, without any risk to the host system.
- **UR-05 (Shared Execution Results):** All project members shall observe the status, standard output, standard error and exit code of every run.
- **UR-06 (Collaborative Breakpoint Debugging):** Users shall set breakpoints, pause, step and inspect variables, with the paused state shared by all project members.
- **UR-07 (Snapshots and Version History):** Members shall capture named snapshots of the project, restore any snapshot, and view and restore earlier versions of individual files.
- **UR-08 (Human-Gated AI Coding Assistance):** Users shall query an AI assistant about the project; any code change it suggests shall require explicit review and approval by a member with editing rights before it is applied.
- **UR-09 (Tiered Project Permissions):** Owners shall assign roles (Viewer, Editor, Debugger, Owner) that control who may edit, execute, debug or administer a project.
- **UR-10 (Fair, Contention-Aware Execution Scheduling):** When more run requests arrive than there is execution capacity, the system shall grant execution capacity by an explicit, deterministic rule rather than by arrival order alone.

1.3 Structured Specifications

Five core transactions are specified below in the structured-specification format (Function, Description, Inputs, Outputs, Operations, Pre-conditions, Post-conditions, Side effects).

SPEC-01: Process a Real-Time Edit	
Description	Accepts an incremental CRDT (Yjs) update from a client over the file's WebSocket collaboration channel, verifies that the sender may edit, applies the update to the server's in-memory CRDT document, and broadcasts it to all other clients connected to the same file.
Inputs	Project identifier, file identifier, authenticated user identity, and the CRDT update (Base64-encoded in a JSON message).
Outputs	The update broadcast to every other client in the file's collaboration room; an acknowledgement to the sender.
Operations	1. Verify that the sender's role is Editor or above; otherwise reject. 2. Locate the file's in-memory CRDT document (its collaboration room). 3. Apply the update under the room lock. 4. Broadcast the update to all other clients in the room. 5. Mark the document as modified so that it is included in the next PostgreSQL checkpoint.
Pre-conditions	The WebSocket connection is authenticated with a valid JWT and the user is a project member.
Post-conditions	The server replica and all connected client replicas converge to the same document state.
Side effects	An update from a Viewer is discarded and an error message is returned to the sender; an update that cannot be decoded is discarded and logged.

SPEC-02: Dispatch a Code Execution	
Description	Validates the request, creates an execution record in the QUEUED state in PostgreSQL, and pushes a job onto the Redis execution queue.
Inputs	Project identifier, file identifier, authenticated user identity, and optional standard input for the program.
Outputs	HTTP 202 (Accepted) with the execution record; a job on the Redis queue; a queued notification to all project members.
Operations	1. Verify that the requester's role is Editor or above. 2. Verify that the file belongs to the project and is in a runnable language. 3. Read the current file text from the live CRDT document. 4. Create the execution record in the QUEUED state. 5. Push the job onto the Redis queue. 6. Notify all project members that the run is queued.
Pre-conditions	The user is authenticated and is a project member.
Post-conditions	Exactly one job exists on the Redis queue for the new execution record.
Side effects	If Redis is unreachable, the request fails with a server error and the committed record remains in QUEUED; automatic reconciliation of such records is tracked as a defect for Sprint 2.

SPEC-03: Step a Debug Session	
Description	Sends a Debug Adapter Protocol (DAP) step command (continue, next, step in, step out) to the debugger running inside the sandbox, and shares the resulting paused state with every project member.
Inputs	Project identifier, execution identifier of the active debug session, and the step command.
Outputs	A DAP request to the debugger; a debug-paused event (current file, line, call stack and variables) broadcast to all project members.
Operations	1. Verify that the requester's role is Debugger or Owner. 2. Verify that the execution is the project's active debug session. 3. Publish the command on the session's Redis command channel. 4. The worker issues the corresponding DAP request to the debugger. 5. On the DAP stopped event, the worker reads the stack trace and variable scopes. 6. The worker publishes a debug-paused event through Redis Pub/Sub to all project members.
Pre-conditions	A debug session is active for the project and the program is paused.
Post-conditions	The program advances as requested and every member sees the same paused location.
Side effects	If the program terminates, a debug-terminated event is broadcast and all paused-line highlights are cleared; a session ends automatically after 600 seconds, or after 180 seconds with no step command.

SPEC-04: Apply an AI-Proposed Edit	
Description	Applies an edit proposed by the AI assistant to a live file, only after the requesting member reviews it and explicitly accepts it.
Inputs	The proposal (target file, exact original text, replacement text, explanation) returned by the assistant.
Outputs	A CRDT update to the target file, broadcast to all clients connected to it.
Operations	1. The assistant returns the proposal without modifying any file. 2. The client displays the proposal as a side-by-side difference. 3. On acceptance, the client applies the replacement to its CRDT document. 4. The resulting CRDT update is processed by SPEC-01, which enforces the Editor role.
Pre-conditions	The original text occurs exactly once in the file at proposal time.
Post-conditions	The change appears for all collaborators and is recorded in the file's change history under the accepting member.
Side effects	If the member declines, the proposal is discarded and the file is unchanged; a Viewer cannot apply a proposal because SPEC-01 rejects the update.

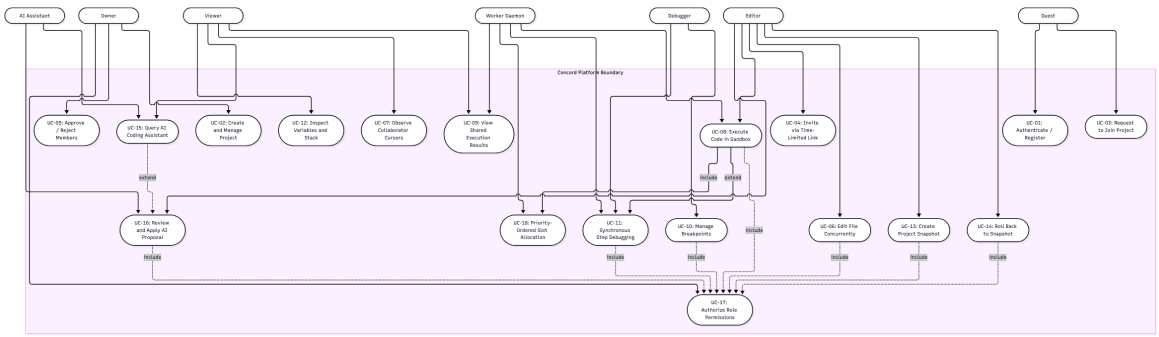
SPEC-05: Allocate a Free Execution Slot	
Description	When an execution slot becomes free, selects which waiting execution receives it using the priority score of Section 5.4 rather than arrival order.
Operations	1. Recompute the priority score of every waiting execution. 2. Select the waiting execution with the highest score (the head of a Redis sorted set). 3. Bind it to the free slot and transition it to STARTING. 4. When its sandbox container exits, release the slot and repeat.
Post-conditions	The executions holding slots are always the highest-priority waiting executions, one per slot; no slot is ever bound to two executions at once.
Status	Specified in this deliverable and tracked as backlog items `CONCORD-36` and `CONCORD-37` (Section 9); implementation is scheduled for Deliverable 2.

2 Use-Case Diagram and User Stories

2.1 Actor Taxonomy

- **Guest (unregistered user):** registers, logs in, or requests to join a project.
- **Viewer:** read-only access; sees files, cursors, execution results and debug sessions, and may query the AI assistant, but cannot modify anything.
- **Editor:** all Viewer rights, plus editing files, running and cancelling executions, applying AI proposals, creating and restoring snapshots, and inviting members at or below the Editor role.
- **Debugger:** all Editor rights, plus starting debug sessions, setting and clearing breakpoints, and stepping.
- **Owner:** all Debugger rights, plus approving join requests, adding and removing members, changing roles, and configuring the AI assistant.
- **Execution Worker** (system actor): consumes jobs from the Redis queue, provisions sandbox containers, enforces resource limits, and publishes results.
- **AI Coding Assistant** (system actor): an external large language model (Mistral or HuggingFace), invoked only by the backend through a fixed set of five tools.

2.2 UML Use-Case Diagram



2.3 User Stories

Each story is written as a narrative scenario: a named user, a concrete situation, the main sequence of events, and the alternative paths the system must handle.

US-01: Joining a Project

Anya is a student joining a group project. She creates an account with her email address and a password; the system validates the email format, stores the password only as a one-way bcrypt hash, and logs her in. The project already exists, so Anya submits a request to join it. The project owner sees the request and approves it with a chosen role (Viewer, Editor or Debugger), after which Anya is a member immediately. Alternatively, a teammate with editing rights could have sent Anya an invitation link carrying a role no higher than the teammate's own; Anya could then join directly through the link, provided it had not passed its seven-day expiry.

Related requirements: FR-01, FR-03, FR-04, FR-05.

US-02: Editing a File Together in Real Time

Priya and Farhan are editing the same file at the same time from different computers. Neither locks the file: each change appears on the other's screen almost instantly, and each can see the other's cursor labelled with a name and a distinct colour. If Farhan's connection drops briefly, his edits are merged with Priya's when he reconnects, and both copies of the file converge to the same text with nothing lost. Rahul, who holds the Viewer role in the same project, watches the edits live; if his client attempted to send an edit, the server would reject it without affecting the shared document.

Related requirements: FR-06, FR-08, FR-09, NFR-P1.

US-03: Running Code Safely

Deepak presses Run on a program he has just written. Every member of the project sees the run move through the queued, starting and running states, and then sees its output, errors and exit code when it finishes. The program never runs on the server itself: it executes in a temporary Docker container that is removed after the run. The container has no network access, a read-only file system, and fixed limits on memory, CPU and number of processes. If the program exceeds its memory limit it is terminated and the run is reported as failed; if it runs longer than ten seconds it is stopped and reported as timed out.

Related requirements: FR-10, FR-11, FR-12, FR-13, NFR-S1, NFR-S2, NFR-SEC1.

US-04: Getting a Fair Turn When Many People Run Code at Once

During a busy laboratory session, ten students press Run within a few seconds of each other, but the system can execute only a fixed number of programs at once. Instead of serving requests strictly in arrival order, the system computes a priority for every waiting request: a request from a member who is actively debugging ranks above an ordinary run, and a request that has waited longer gains priority steadily, so that no request waits indefinitely. A request that arrived later may therefore run earlier if its priority is higher. Whenever a running program finishes, the free slot is given to the waiting request with the highest priority at that moment, and no slot is ever given to two runs at once.

Related requirements: FR-14, FR-15, NFR-D5.

US-05: Debugging Together

Meera is looking for a bug in a Python program. She sets a breakpoint and starts a debug session. When execution reaches the breakpoint, every project member sees the same paused line, call stack and local variables without any action of their own. When Meera steps to the next line, all views update together. Any other member with the Debugger or Owner role may also issue step commands in the same session, and every member may expand variables to inspect them, but only one debug session may be active in a project at a time, so the team always works on a single paused program. A member who opens the project mid-session is shown the current paused location immediately.

Related requirements: FR-17, FR-18, NFR-P3.

US-06: Saving and Restoring Work

Before a risky change, Owen takes a named snapshot of the whole project. After an hour of changes across several files, the team decides to revert. Owen selects the earlier snapshot and restores it. The system first takes a safety snapshot of the current state, then replaces every file with its saved content, and every member's editor updates to the restored version without reloading. A Viewer attempting the same restore would be refused and no file would change.

Related requirements: FR-19, FR-25.

US-07: Asking the AI Assistant for Help, With a Human in Control

Leo's program fails and he does not understand the error. He asks the assistant to explain it. The assistant reads his file and the output of his last run and replies with an explanation; in this explain mode it has no ability to change code. Later, Leo asks it to fix a different bug. The assistant reads the relevant file, prepares a change, and returns it as a proposal shown beside the original code. Leo either accepts it, in which case it is applied to the shared file as an ordinary edit that everyone sees, or declines it, in which case nothing changes. The assistant cannot modify any file without a member reviewing and accepting the change.

Related requirements: FR-20, FR-21, FR-22, NFR-EXT-ETH1.

3 Functional and Non-Functional Requirements

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Users shall sign up and log in with email and password. Passwords shall be stored as bcrypt hashes. On success the system shall issue an HS256-signed JWT, sent in the `Authorization` header for REST requests and as a query parameter when opening a WebSocket connection.	Mandatory
FR-02	A user shall create a project and become its Owner.	Mandatory
FR-03	A member with the Editor role or above shall generate an invitation link that carries a target role no higher than the inviter's own role and expires after seven days.	Mandatory
FR-04	A non-member shall submit a join request; the Owner shall approve it with a chosen role or reject it. Duplicate pending requests shall be refused.	Mandatory

ID	Requirement	Priority
FR-05	The Owner shall add members, change their roles and remove them; the Owner's own role cannot be changed or removed.	Mandatory
FR-06	The system shall enforce a four-level role hierarchy (Viewer, Editor, Debugger, Owner, each including the rights of the level below) on every REST request and on every inbound WebSocket edit message.	Mandatory
FR-07	Every member shall see the project's file tree; an Editor or above shall create, rename, move and delete files and folders.	Mandatory
FR-08	Editors shall edit a file concurrently; edits shall converge through a Conflict-free Replicated Data Type (CRDT) so that no edit is lost regardless of arrival order.	Mandatory
FR-09	Connected members shall see the live cursor position, name and colour of every other member in the same file.	Mandatory
FR-10	An Editor or above shall run a file. The run shall follow a finite state machine: QUEUED, STARTING, an optional COMPILING stage, RUNNING, and one terminal state among COMPLETED, FAILED, TIMEOUT and CANCELLED. Only transitions listed in the allowed-transitions table shall be permitted; terminal states have no outgoing transitions.	Mandatory
FR-11	Supported languages shall be Python, JavaScript (Node.js) and C++. C++ shall be compiled before running; a compilation failure shall move the run to FAILED without executing it. A run may be given standard input.	Mandatory
FR-12	Every status transition shall be broadcast to all project members in real time; on completion, the standard output, standard error, exit code and duration shall be broadcast to all members and stored with the execution record.	Mandatory
FR-13	An Editor or above shall cancel a queued or running execution; the worker shall stop the sandbox container and record the run as CANCELLED.	Mandatory
FR-14	Run requests shall be pushed onto a Redis queue and consumed by a bounded pool of worker processes, each running one execution at a time. **This bounded pool of execution slots is the finite, contended resource** (Section 5.4).	Mandatory
FR-15	When more executions are waiting than there are free slots, the next free slot shall be granted by a computed **priority score**, not by arrival order. **Status: specified in Deliverable 1, implemented in Deliverable 2**; the current Redis queue is first-in-first-out (FIFO).	Mandatory
FR-16	A background sweeper shall run every 60 seconds and mark as FAILED any execution that has remained in STARTING, COMPILING or RUNNING for more than five minutes, and shall signal its worker to stop the container.	Mandatory
FR-17	A Debugger or Owner shall start a debug session for a Python file, set and clear breakpoints, and control execution (continue, next, step in, step out) through the Debug Adapter Protocol (DAP).	Mandatory
FR-18	At most one debug session shall be active per project. Every member shall see the same paused state and may inspect variables; only Debuggers and Owners may issue step commands.	Mandatory
FR-19	An Editor or above shall create a named snapshot of the whole project and restore it; restoring shall first take a safety snapshot and then replace the live content of every file for all connected clients.	Mandatory
FR-20	The AI assistant shall support two large language model (LLM) providers, Mistral (Codestral) and HuggingFace, behind a common provider interface. Only the Owner may configure a project's provider and API key; the key shall be stored encrypted.	Mandatory
FR-21	Any member shall query the AI assistant in one of five modes (ask, explain, review, fix, generate). The assistant shall run only on the backend, may take at most six tool-calling steps, and may use only five tools: list project files, read a file, read the result of the last run, check whether code parses, and propose an edit. The explain and review modes shall have no access to the propose-edit tool. A proposed edit shall never be written by the assistant; it shall be applied only when a member accepts it (SPEC-04).	Mandatory
FR-22	Every AI request shall be recorded with its mode, prompt, response, provider, model, token counts, latency and outcome.	Mandatory
FR-23	The system shall expose execution metrics (status summary, runs in the last hour, recent events) on a metrics page.	Optional
FR-24	AI requests shall be rate-limited per project (ten requests per 60-second window), with the limit stored in Redis.	Mandatory
FR-25	Each file shall keep a version history; a member shall view differences between versions, and an Editor or above shall restore an earlier version.	Mandatory

ID	Requirement	Priority
FR-26	Members shall search text across all files of a project.	Optional

3.2 Non-Functional Requirements

Product Requirements: Usability

- **NFR-U1:** The client shall run in current Chrome, Firefox, Safari and Edge browsers with no plugins or local installation.
- **NFR-U2:** Each collaborator shall be identified by a coloured, named cursor and selection.
- **NFR-U3:** Compilation and runtime errors shall be shown in the console with readable line references.
- **NFR-U4:** The interface shall follow the conventional IDE layout (file tree, tabbed editor, console, side panels), so that a new student can run code within three minutes of first login.

Product Requirements: Efficiency (Performance)

- **NFR-P1:** With ten concurrent editors, 99% of CRDT updates shall reach all other clients in under 300 ms (target under 100 ms).
- **NFR-P2:** An execution shall move from QUEUED to STARTING in under 1.0 s (95th percentile) with up to five simultaneous run requests.
- **NFR-P3:** A debug step shall be reflected on all collaborators' screens in under 150 ms.
- **NFR-P4:** The editor shall render at 60 frames per second for the typing user.

Product Requirements: Efficiency (Space)

- **NFR-S1:** Each sandbox container shall be limited to 128 MB of memory; a container that exceeds it shall be terminated.
- **NFR-S2:** Each sandbox container shall be limited to 0.5 CPU.
- **NFR-S3:** The production client bundle shall not exceed 2.5 MB compressed.
- **NFR-S4:** Stored file versions shall be pruned periodically so that version history does not grow without bound.

Product Requirements: Dependability

- **NFR-D1:** The service shall be available at least 99.5% of scheduled laboratory hours.
- **NFR-D2:** A backend crash shall lose at most the last 5-second checkpoint interval of edits.
- **NFR-D3:** All CRDT replicas shall converge to identical content under any ordering of updates.
- **NFR-D4:** Every execution orphaned by a worker failure shall be marked FAILED within six minutes of its start (five-minute threshold plus one 60-second sweep interval).
- **NFR-D5:** At any instant the executions holding slots shall be exactly the N highest-priority waiting executions, and **no slot shall ever be assigned twice**; this is verified by the Deliverable 3 concurrency experiment.

Product Requirements: Security

- **NFR-SEC1:** Sandbox containers shall run as a non-root user (UID 10001) with networking disabled, a read-only root file system and the project workspace mounted read-only. Dropping all Linux capabilities is a required hardening item scheduled for Sprint 2.
- **NFR-SEC2:** Passwords shall be stored only as bcrypt hashes and shall never be logged.
- **NFR-SEC3:** The server shall prevent Viewers from modifying code, running programs or changing settings, independently of the client.
- **NFR-SEC4:** AI tools shall only access files that belong to the requesting project.

Organizational Requirements

- **NFR-ORG-ENV1:** The full system shall be containerized and orchestrated with Docker Compose.
- **NFR-ORG-ENV2:** Sandbox images shall provide Python 3.12, Node.js 20 and a C++17 compiler (g++ on Ubuntu 22.04).
- **NFR-ORG-ENV3:** PostgreSQL 16 shall be used for persistent storage and Redis 7 (with append-only persistence) for the execution queue and Pub/Sub messaging.
- **NFR-ORG-OP1:** The backend shall expose a health-check endpoint; extending it to report PostgreSQL and Redis connectivity and Redis queue depth is scheduled for Sprint 2.
- **NFR-ORG-OP2:** On graceful shutdown the backend shall force a checkpoint of all modified documents to PostgreSQL before closing connections.
- **NFR-ORG-OP3:** Execution state changes shall be recorded in an append-only execution-event log.
- **NFR-ORG-DEV1:** Continuous integration (GitHub Actions) shall run linting, static type checking, static security analysis, unit tests, a frontend build, and an end-to-end test job on every pull request.
- **NFR-ORG-DEV2:** Backend line coverage shall reach at least 80%; the CI gate is currently set at 78% and

will be raised to 80% by Deliverable 3.
- **NFR-ORG-DEV3:** Work shall be tracked in Jira with story-point estimates and a Definition of Done.

External Requirements
- **NFR-EXT-REG1:** All external traffic shall use TLS (HTTPS and WSS), terminated by a reverse proxy; no personal data beyond email address and username shall be stored.
- **NFR-EXT-REG2:** JWTs shall expire after 24 hours.
- **NFR-EXT-ETH1:** The AI assistant shall never modify a workspace without explicit human acceptance.
- **NFR-EXT-ETH2:** AI requests shall be retained so that instructors can audit the level of AI assistance; accepted proposals appear in file change history under the accepting member.
- **NFR-EXT-LEG-ACC1:** Each execution shall record its queue, start and finish times, exit code and worker; recording CPU time and peak memory is planned for Deliverable 3.
- **NFR-EXT-LEG-ACC2:** Token usage per AI request shall be recorded to bound external API cost.
- **NFR-EXT-LEG-SEC1:** Sandbox containers shall have no network access to the host, cloud metadata services or the internet.
- **NFR-EXT-LEG-SEC2:** Each sandbox container shall be limited to 64 processes, preventing fork bombs.

3.3 Metrics for Non-Functional Requirements

Quality Property	**Metric**	**Target / Verification**
Speed	API throughput; CRDT propagation latency; screen refresh time	at least 200 requests/s; under 300 ms (99th percentile); at most 16.6 ms
Size	Memory per sandbox; client bundle size	at most 128 MB; at most 2.5 MB compressed
Ease of use	Training time	under 3 minutes (laboratory user test)
Reliability	Mean time between failures; availability	over 720 h; 99.5% (health check)
Robustness	Time to restart after failure; slot double-assignment rate	under 5.0 s; **0** (Deliverable 3 concurrency experiment, NFR-D5)
Portability	Number of target systems	3 (macOS, Linux, Windows through Docker)

3.4 Requirements Validation
1. **Validity:** CRDT-based editing, sandboxed execution, collaborative debugging and priority-scheduled execution address the needs of the stated stakeholders, including defined behaviour under contended execution capacity.
2. **Consistency:** Document distribution (all room members receive updates) is separated from mutation authorization (edits are checked against role on every message).
3. **Completeness:** Failure handling is specified for worker failure (FR-16), memory exhaustion (NFR-S1), timeouts (FR-10), cancellation (FR-13) and slot contention (FR-15, NFR-D5).
4. **Realism:** In-memory CRDT operations take under 2 ms and Redis Pub/Sub delivery under 5 ms; a Redis sorted-set insertion is $O(\log n)$ for a laboratory-scale queue.
5. **Verifiability:** Every functional requirement maps to a planned test (Section 4); FR-15 and NFR-D5 map to the Deliverable 3 concurrency experiment.

4 Initial Requirements Traceability Matrix

4.1 User Requirement to Functional Requirement, Use Case and Specification

UR	FRs	UCs	Spec
UR-01	FR-01	UC-01	None
UR-02	FR-02, FR-07	UC-02	None
UR-03	FR-08, FR-09	UC-06, UC-07	SPEC-01
UR-04	FR-10, FR-11, FR-13, FR-16	UC-08	SPEC-02
UR-05	FR-12	UC-09	None
UR-06	FR-17, FR-18	UC-10, UC-11, UC-12	SPEC-03
UR-07	FR-19, FR-25	UC-13, UC-14	None
UR-08	FR-20, FR-21, FR-22, FR-24	UC-15, UC-16	SPEC-04
UR-09	FR-03, FR-04, FR-05, FR-06	UC-03, UC-04, UC-05, UC-17	None
UR-10	FR-14, FR-15	UC-18	SPEC-05

4.2 Critical Requirements to Jira Issue and Planned Test

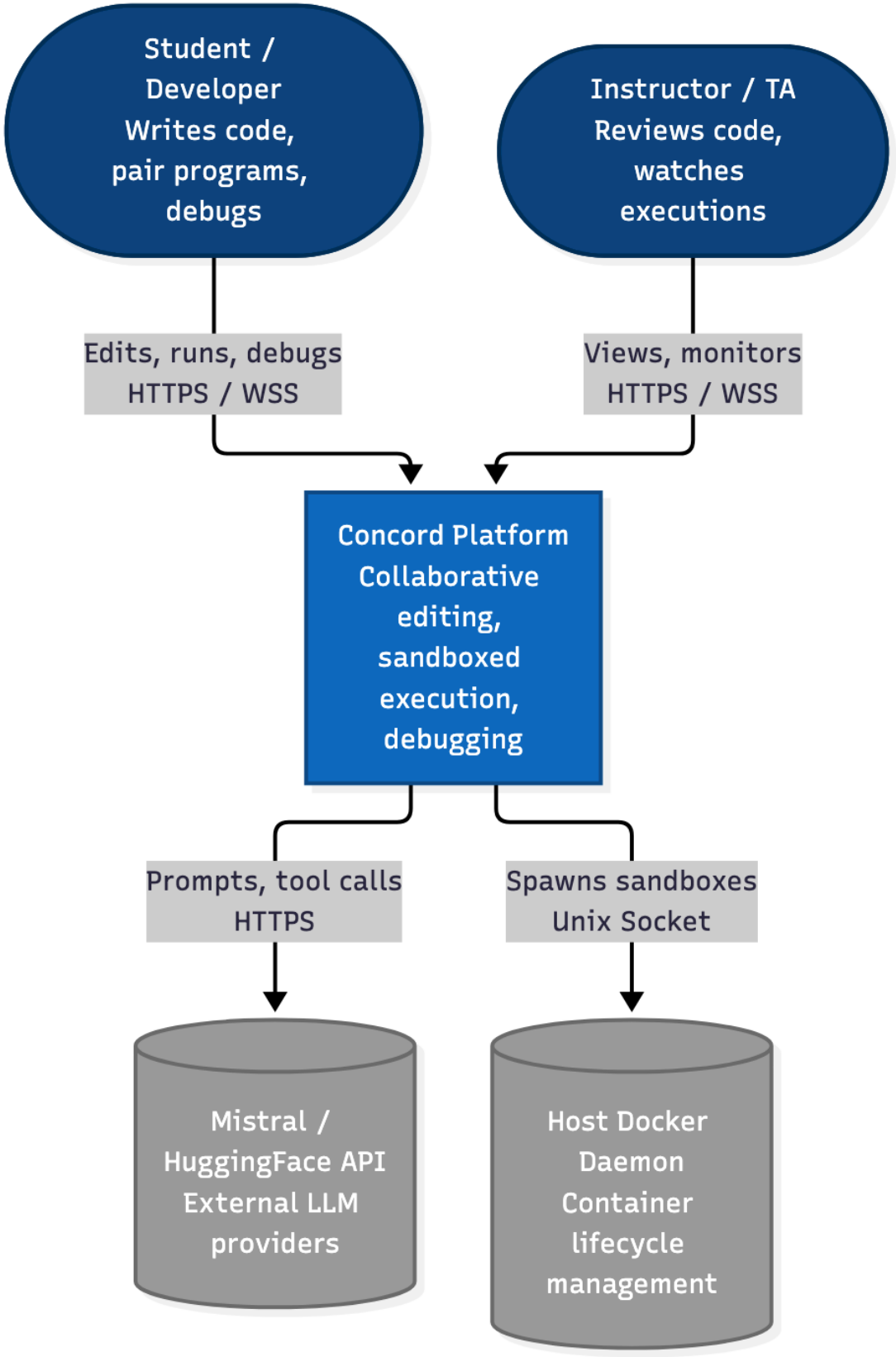
Requirement	Story	Jira Issue	Planned Test (Deliverable 3)
FR-01	US-01	`CONCORD-2`, `CONCORD-3`	Unit and integration tests of signup, login and token validation
FR-03, FR-04	US-01	`CONCORD-24`	Integration tests of invitation links and join requests
FR-08, NFR-P1	US-02	`CONCORD-7`, `CONCORD-8`	Concurrency test of simultaneous edits from ten or more clients
FR-10, FR-11	US-03	`CONCORD-13`	Unit tests of the execution state machine, plus system tests
FR-14, FR-15	**US-04**	`CONCORD-36`, `CONCORD-37`	**Concurrency test proving no slot is assigned twice**
FR-16	US-03	`CONCORD-13`	Failure test: worker terminated during a run
FR-17, FR-18	US-05	`CONCORD-16`, `CONCORD-17`	DAP client tests and a multi-client debug session test
FR-21, FR-22	US-07	`CONCORD-20`, `CONCORD-21`	Tool-contract unit tests and adversarial tests of agent output
FR-06	US-02	`CONCORD-24`	Authorization-abuse tests across all four roles

4.3 Functional Subsystem to Governing Non-Functional Requirements

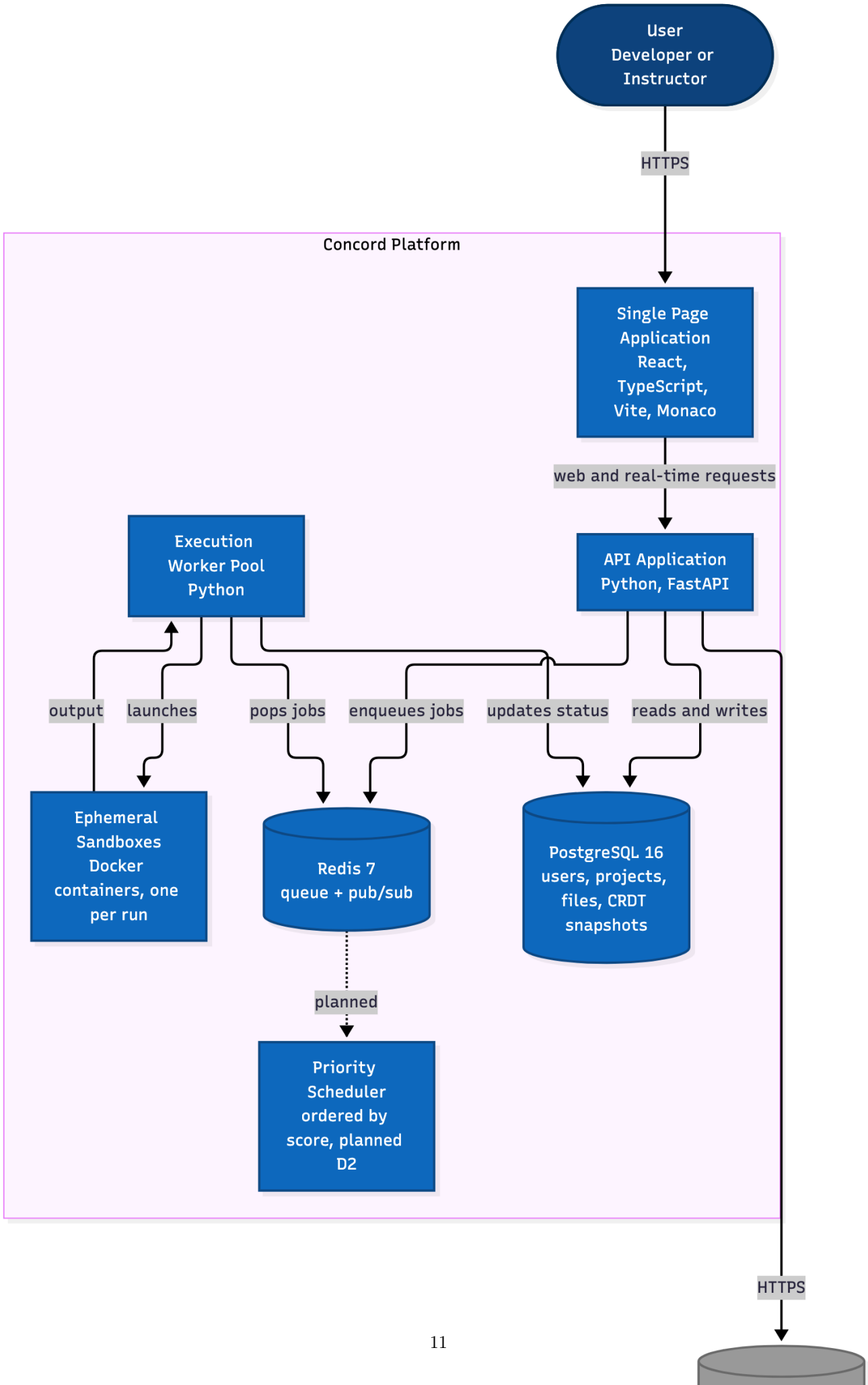
Subsystem	FRs	Governing NFRs
Authentication and authorization	FR-01, FR-03 to FR-06	NFR-SEC2, NFR-SEC3, NFR-EXT-REG1, NFR-EXT-REG2
Collaborative editor (CRDT)	FR-08, FR-09, FR-25	NFR-P1, NFR-P4, NFR-D2, NFR-D3, NFR-ORG-OP2, NFR-S4
Sandboxed execution	FR-10 to FR-13, FR-16	NFR-P2, NFR-S1, NFR-S2, NFR-D4, NFR-SEC1, NFR-EXT-LEG-SEC1, NFR-EXT-LEG-SEC2, NFR-EXT-LEG-ACC1
Execution slot allocation	**FR-14, FR-15**	**NFR-D5**
Collaborative debugger	FR-17, FR-18	NFR-P3
AI assistant	FR-20 to FR-22, FR-24	NFR-SEC4, NFR-EXT-ETH1, NFR-EXT-ETH2, NFR-EXT-LEG-ACC2
Client and storage	FR-02, FR-07, FR-26	NFR-U1 to NFR-U4, NFR-S3, NFR-D1

5 Architecture and Design Document

5.1 System Context



5.2 Container Diagram (System Architecture)



The system follows a client-server architecture. A React single-page application (Monaco editor, Yjs CRDT) communicates with a FastAPI backend over HTTPS and WebSockets. The backend persists data in PostgreSQL and uses Redis both as the execution queue and as the Pub/Sub bus between backend processes and execution workers. Workers launch one Docker container per run on the host's Docker daemon. The backend calls the configured external LLM provider over HTTPS.

5.3 Components

Component	Responsibility	Layer
REST API	Authentication, projects, members, invitations, files, folders, executions, snapshots, debugging, AI assistant and metrics endpoints.	Backend
Real-Time Gateway	Two WebSocket channels: a project channel (presence, execution, debug and AI progress events) and a per-file collaboration channel (CRDT updates).	Backend, real-time
Collaboration Room Manager	Holds one in-memory CRDT document per open file, applies and broadcasts updates, and checkpoints modified documents to PostgreSQL every 5 s.	Backend, real-time
Execution Queue and Scheduler	Redis list used as a FIFO job queue plus Redis Pub/Sub for events; priority ordering is added in Deliverable 2 (FR-15).	Backend, real-time
Orphan Sweeper	Background task that fails executions stuck beyond the five-minute threshold (FR-16).	Backend
Execution Worker Pool	Consumes jobs, builds and runs sandbox containers, publishes status and results, and records the final state.	Worker service
Sandbox	One Docker image per language with hard resource limits; one short-lived container per run.	Sandbox images
Debug Coordinator	Runs the DAP session inside the worker and relays commands and paused state through Redis.	Worker and backend
LLM Manager (AI Gateway)	Provider-independent interface over Mistral and HuggingFace; encrypts stored API keys; applies per-project rate limits.	Backend, AI
Agent Runner	Bounded tool-calling loop over the five assistant tools.	Backend, AI
Role Guard	Resolves the caller's project role and compares it with the minimum role required by each operation.	Backend, core

5.4 Finite Resource and Calculation Rule

Finite resource. The execution tier has a fixed pool of N **execution slots**, equal to the number of worker processes, since each worker runs one execution at a time ($N = 2$ in the reference deployment). A slot is acquired when a worker takes a job and starts its sandbox, and released when the sandbox exits. The Redis blocking pop (`BRPOP`) delivers each job to exactly one worker, so two executions can never hold the same slot. Unlike the shared document, which is an unbounded, mergeable resource, a slot is an exclusive unit that cannot be allocated twice.

Calculation rule: execution priority score. When more executions are waiting than there are free slots, the next slot is granted using a score computed for every waiting execution e:

$$\text{priority}(e) = w_{\text{role}} \cdot \text{roleWeight}(e) + w_{\text{wait}} \cdot \min\left(\frac{\text{waitTime}(e)}{30}, 1\right) - w_{\text{lang}} \cdot \text{compileCost}(e)$$

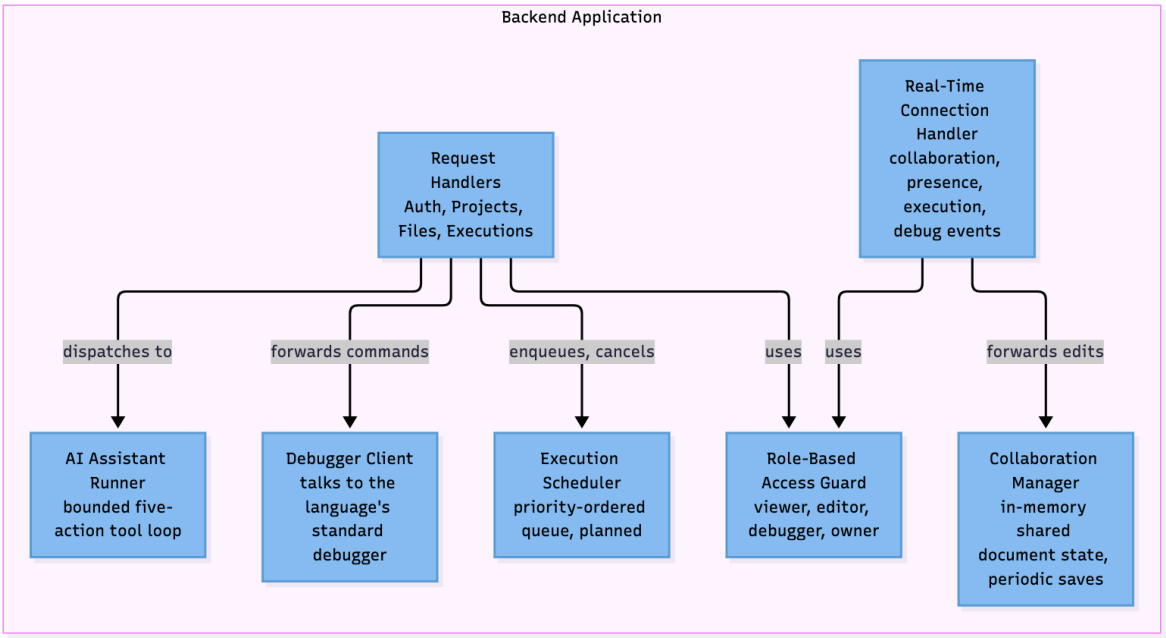
- *Role weight:* Owner highest, then Debugger, Editor and Viewer; a debug run receives additional weight so that an interactive session is not starved.
- *Wait-time term:* increases linearly and saturates after 30 seconds; this ageing term guarantees that no request waits indefinitely, regardless of role.
- *Compilation cost term:* a language that requires compilation (C++) is weighted down slightly under contention, because it occupies its slot longer.

The scheduler will replace the current FIFO Redis list with a Redis sorted set ordered by priority score, re-scored on each arrival and each scheduling decision. This gives Deliverable 3 a testable correctness property: *the N executions holding slots are always the N highest-priority waiting executions, and no slot is ever assigned twice.*

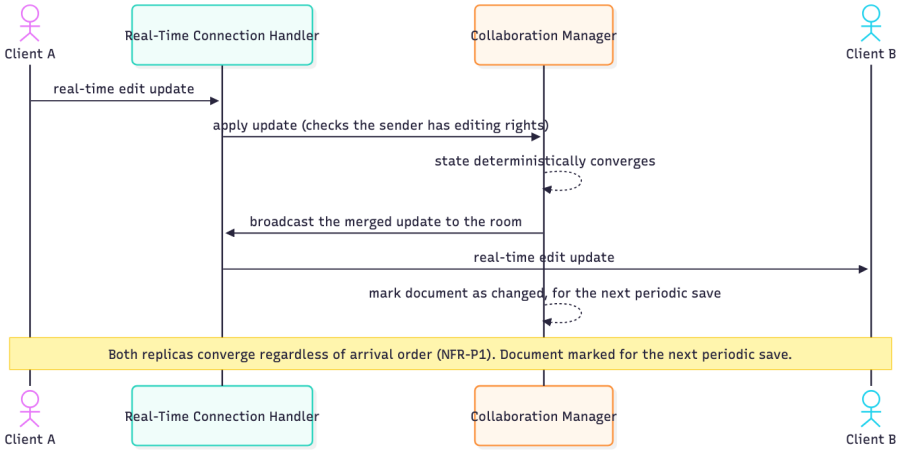
Status: specified here and tracked as `CONCORD-36` and `CONCORD-37` (Section 9); implementation is scheduled for Deliverable 2.

6 Component and Sequence Diagrams

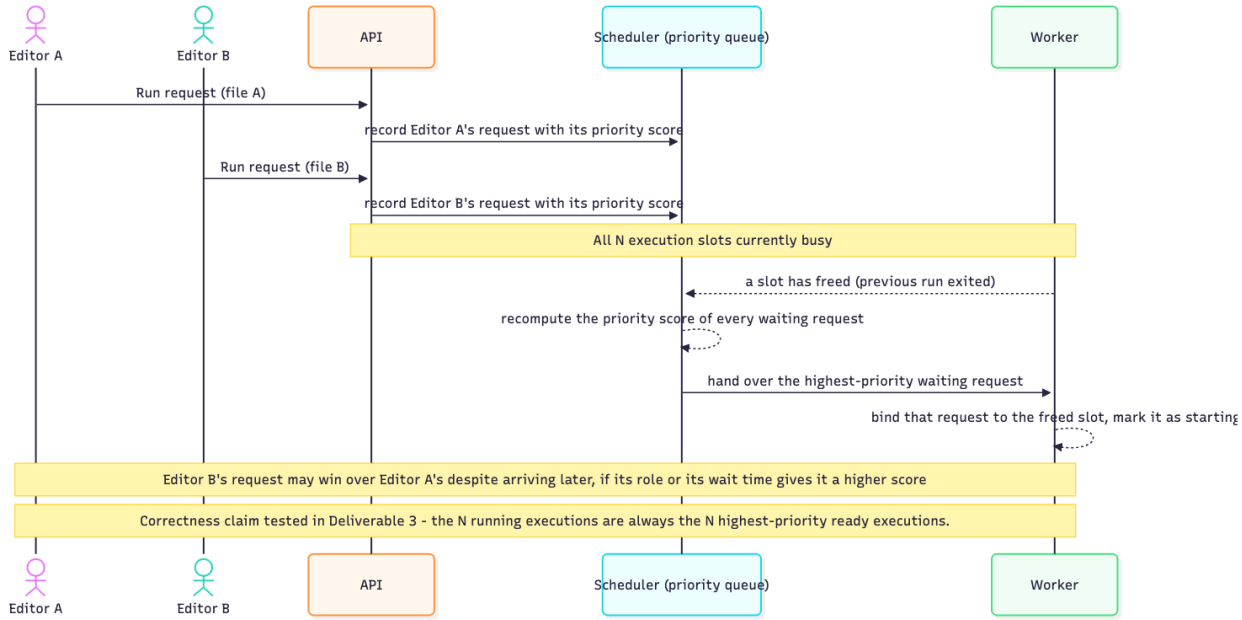
6.1 Component Diagram



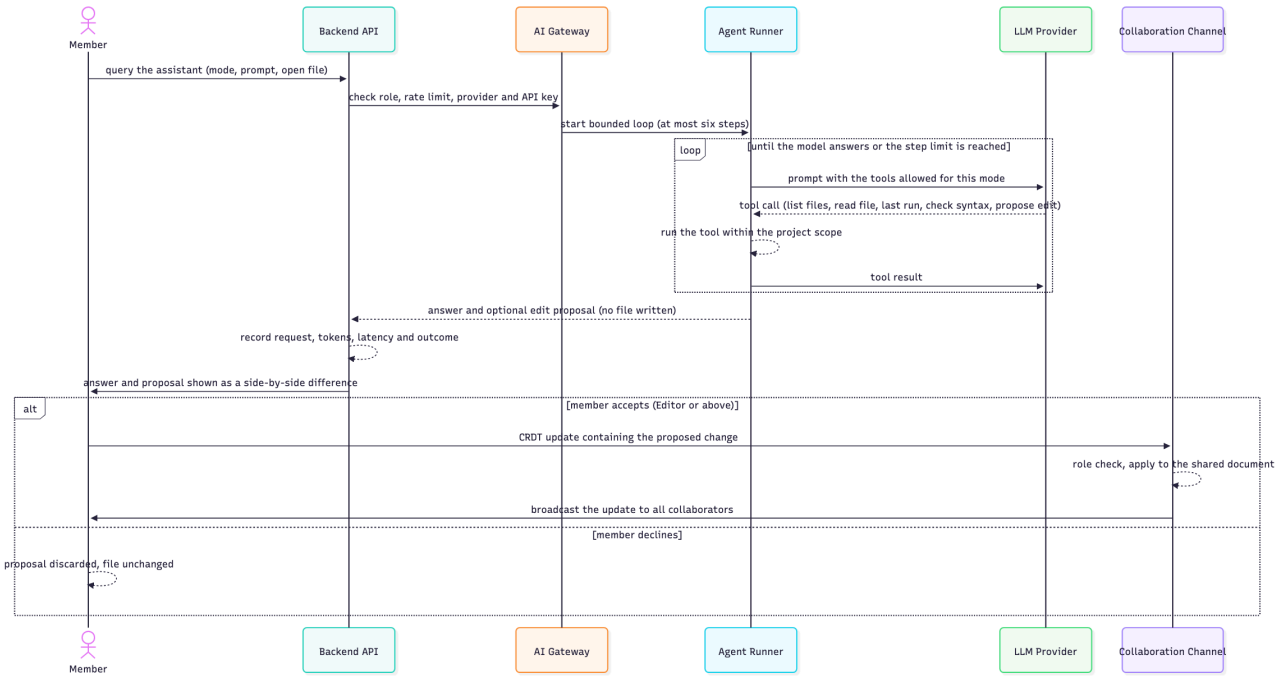
6.2 SD-1: Real-Time CRDT Edit Flow



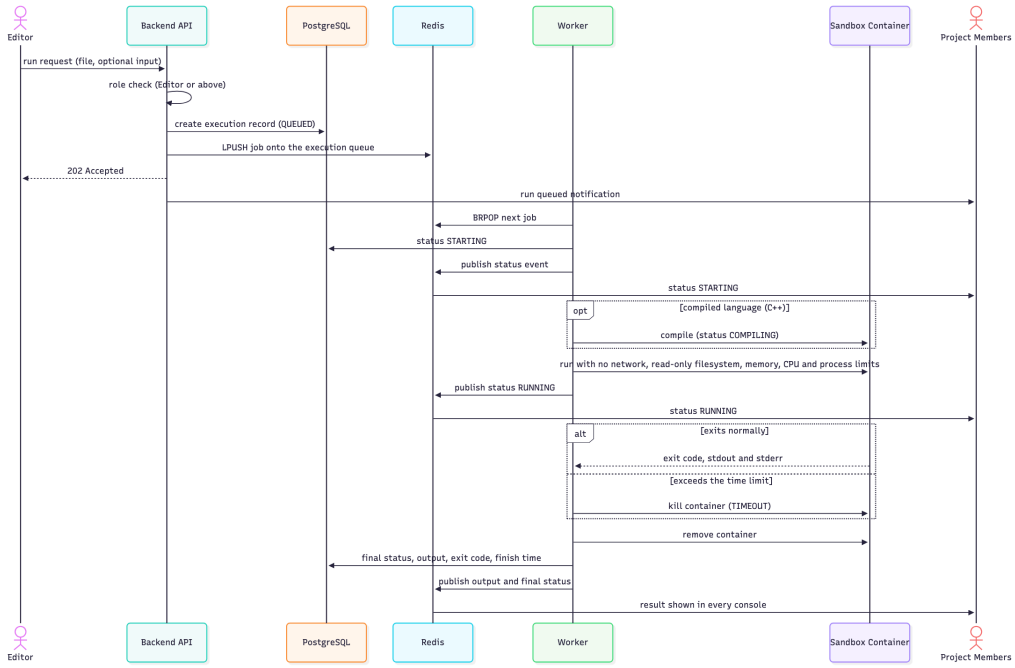
6.3 SD-2: Priority-Ordered Execution Slot Allocation



6.4 SD-3: Human-Gated AI Assistant Flow



6.5 SD-4: Sandboxed Execution and Result Broadcast



7 Design-Pattern Mapping

7.1 General Design Patterns

Pattern	Application and Rationale	Status
Client-Server (architectural)	Browser clients use services provided by the backend, which in turn uses PostgreSQL, Redis and the worker tier.	Implemented
Layered (architectural)	The backend separates request handling, domain logic (real-time, AI, debugging) and persistence.	Implemented
State	The execution lifecycle is a finite state machine with a single table of allowed transitions, so an illegal transition such as COMPLETED to RUNNING cannot occur.	Implemented
Observer (Publish/Subscribe)	Clients are notified of edits, execution and debug events without polling: an in-process broadcaster serves clients of the same backend process, and Redis Pub/Sub serves clients connected to other processes.	Implemented
Adapter	A DAP client wraps the debugger's message-framed protocol over a TCP socket behind simple operations (set breakpoints, continue, next, step in, step out), isolating the rest of the system from protocol details.	Implemented
Strategy	The LLM integration is written against one abstract provider interface with one concrete implementation per provider (Mistral, HuggingFace).	Implemented
Strategy (deferred)	Build and run steps are selected per language from a runtime registry; extracting them into one strategy class per language is planned.	Deferred
Singleton	The collaboration room manager and the application configuration are instantiated once and shared across the application.	Implemented
Dependency Injection (session per request)	Each request receives its own asynchronous database session through the web framework's dependency injection.	Implemented
Role Guard	One centralized check resolves the caller's project role and compares it with the minimum role required, applied uniformly to REST requests and WebSocket edit messages.	Implemented

7.2 AI-Specific Architectural Patterns

The AI assistant additionally uses patterns specific to AI-based systems, since it must manage uncertain output, tool autonomy and human-approval boundaries.

AI Pattern	Application and Rationale	Status
AI Gateway	One component selects the project's configured provider, decrypts its API key and applies rate limiting before any request reaches the provider.	Implemented
Tool-Using Agent	The assistant is limited to five declared tools, each scoped to the current project; tool failures are returned to the model as messages so that it can correct itself.	Implemented
Agent Loop	The assistant follows a bounded plan, act, observe, decide cycle with a hard limit of six steps and live progress updates to the user.	Implemented
Human-in-the-Loop with Guardrail	The only tool that can affect code requires the text to be replaced to occur exactly once, checks whether the result still parses, and only records a proposal; a member must accept it before it is applied.	Implemented
AI Observability	Every request is recorded with mode, provider, model, token counts, latency and errors.	Implemented
Continuous AI Evaluation	A repeatable evaluation set for answer quality does not yet exist.	**Gap, planned for Deliverables 2 and 3**
Fallback and Graceful Degradation	On provider failure the assistant reports the error rather than falling back to another model.	**Gap, planned for Deliverables 2 and 3**

Retrieval-Augmented Generation (RAG) and multi-agent architectures were deliberately not used. The assistant reads the live project files directly through its tools, so a separate retrieval index would add a layer without benefit. A single assistant role does not justify the coordination overhead of several cooperating agents.

8 Architecture Alternatives and Rejected Decisions (ADRs)

ADR	Decision, Alternatives Rejected and Trade-offs
ADR-001: CRDT over Operational Transformation	Chose a CRDT (Yjs in the browser, pycrdt on the backend, sharing one binary update format) for deterministic, lock-free convergence. Rejected: Operational Transformation (requires a central sequencing server); last-write-wins (loses concurrent edits); Automerge (larger update format for this use). Trade-off: document state is stored in PostgreSQL as an opaque binary blob and cannot be queried directly with SQL.
ADR-002: Redis Queue over a Task-Queue Framework	Chose a Redis list as a FIFO job queue (`LPUSH` and `BRPOP`) and Redis Pub/Sub for events, reusing the Redis instance already required. Rejected: Celery with RabbitMQ, and Kafka (operational overhead disproportionate to the system's scale); direct WebSocket connections from the API to each worker (short-lived workers are hard to address). Trade-off: no built-in dead-letter queue or delivery acknowledgement; a job can be lost if Redis fails before append-only persistence writes it.
ADR-003: Periodic PostgreSQL Checkpoints over Per-Keystroke Writes	Live CRDT documents are kept in backend memory and checkpointed to PostgreSQL every 5 seconds, with a forced checkpoint on graceful shutdown. Rejected: writing every CRDT update to PostgreSQL; PostgreSQL LISTEN/NOTIFY for fan-out. Trade-off: up to 5 seconds of edits can be lost on an abrupt crash.
ADR-004: Sibling Docker Containers over Docker-in-Docker	The worker mounts the host's Docker socket and starts each run as a sibling container. Rejected: Docker-in-Docker (requires a privileged outer container); gVisor, Firecracker and nsjail (complexity not justified at this scale). Trade-off: the worker has root-equivalent access to the Docker daemon, mitigated by issuing only constrained run commands.
ADR-005: Stateless JWT over Server-Side Sessions	Chose HS256-signed JWTs, sent in the `Authorization` header for REST and as a query parameter for WebSockets. Rejected: server-side sessions with HttpOnly cookies (unreliable across the WebSocket upgrade); an external OAuth2/OIDC provider (unnecessary dependency). Trade-off: a JWT cannot be revoked before expiry, mitigated by a 24-hour lifetime.

ADR	Decision, Alternatives Rejected and Trade-offs
ADR-006: Two LLM Providers behind One Interface	Mistral (Codestral) and HuggingFace are both implemented behind a common provider interface, with Mistral as the default. Rejected: OpenAI GPT-4o (paid billing is a barrier for an academic deployment); a locally hosted model through Ollama (requires GPU hardware beyond budget). Trade-off: both providers can rate-limit under load, mitigated by per-project API keys and per-project request limits.
ADR-007: Weighted, Ageing Priority Score over a Fixed Priority	Priority combines a role weight, a wait-time term and a compilation-cost penalty. Rejected: a priority fixed at submission time and a role-only weight (both allow higher-role users to starve lower-role users under sustained load). Trade-off: the three weights require tuning, planned for Deliverable 2.
ADR-008: One Container per Run over a Long-Lived Container per Project	Each run gets its own container, removed after the run. Rejected: a shared, long-lived container per project (faster start-up, but a compromised or resource-exhausting run would affect later runs). Trade-off: container start-up adds latency to every run.

9 Jira Project with Initial Backlog

Project key: `CONCORD`. Two-week sprints over an eight-week (four-sprint) lifecycle.

9.1 Epics

1. `EPIC-1` Authentication and Access Control
2. `EPIC-2` Real-Time CRDT Engine
3. `EPIC-3` Project and File System Management
4. `EPIC-4` Containerized Execution Pipeline
5. `EPIC-5` Collaborative Debugging
6. `EPIC-6` Workspace Snapshots and History
7. `EPIC-7` Human-Gated AI Assistant
8. `EPIC-8` DevOps and Infrastructure
9. `EPIC-9` Security and Auditing
10. `EPIC-10` Application Performance Monitoring
11. `EPIC-11` User Interface Design and Accessibility
12. `EPIC-12` End-to-End and Load Testing
13. `EPIC-13` **Execution Priority Scheduling:** finite-resource slot allocation and its calculation rule.

9.2 Backlog (excerpt; the full backlog of 37 issues is maintained in Jira)

Issue	Summary	Epic
`CONCORD-1`	Design and initialize the PostgreSQL schema with Alembic migrations.	EPIC-8
`CONCORD-2`, `3`	User signup and login; JWT validation for REST and WebSocket connections.	EPIC-1
`CONCORD-7`, `8`	Integrate the backend CRDT library; propagate edits in under 300 ms.	EPIC-2
`CONCORD-11` to `14`	Sandbox images; Redis execution queue; sandboxed execution; result broadcast through Redis Pub/Sub.	EPIC-4
`CONCORD-16` to `18`	DAP client; breakpoints and stepping; shared paused state.	EPIC-5
`CONCORD-20`, `21`	LLM tool-calling agent loop; AI edits as reviewable proposals.	EPIC-7
`CONCORD-24`	Enforce the Viewer, Editor, Debugger and Owner roles.	EPIC-1
`CONCORD-36`	**Replace the FIFO Redis list with a priority-ordered Redis sorted set.**	**EPIC-13**
`CONCORD-37`	**Implement and unit-test the priority score; concurrency test proving no slot is assigned twice.**	**EPIC-13**

10 Jira and GitHub Integration

Status: configured.

1. **GitHub application:** the GitHub for Jira application is installed in the Jira workspace.
2. **Repository linked:** `aamirepe/CS455_COURSE_PROJECT`.

3. **Smart Commits:** enabled, so that commit messages can transition issues and log time.
4. **Development panel:** branch status, pull request approvals and CI status are shown on each Jira issue.
5. **Branch protection:** the `main` and `develop` branches require a linked Jira issue in the pull request title and a passing CI run.

Branching strategy (GitFlow):

- `main`: production-ready code, tagged with semantic versions.
- `develop`: integration branch for the current sprint, created from `main`.
- `feature/CONCORD-{ID}-description` and `bugfix/CONCORD-{ID}-description`: created from `develop`.
- `hotfix/CONCORD-{ID}-description`: created from `main`.

Conventions: every branch name, commit message and pull request title carries its Jira issue key; every pull request describes its change and its testing and is reviewed by at least one other team member before merging.

Action item for Sprint 1: pull requests merged before the integration was enabled are to be linked retrospectively to their Jira issues, and issue keys are required on all branches and commits from this point onward. Deliverables are also stored in the repository's `Deliverables` folder.

Access: the instructor and the assigned teaching assistants are added to the Jira project and the GitHub repository with at least read access (owner: Aamir).

11 Initial Sprint Plan

Sprint	Focus	Window	Milestone
1	Foundation, data model, Jira and GitHub traceability, priority-scheduling design	2026-09-26 to 2026-10-10	Schema, authentication and CI in place; priority-scheduling design reviewed
2	Real-time collaboration and security	2026-10-10 to 2026-10-24	CRDT gateway at ten or more users; rate limiting; access control; sandbox hardening
3	Execution, debugging, QA and **priority scheduler**	2026-10-24 to 2026-11-07	Sandboxed execution; DAP debugging; end-to-end tests; `CONCORD-36` **and** `CONCORD-37` **complete**
4	AI assistant, snapshots, metrics and load testing	2026-11-07 to 2026-11-21	AI assistant integrated; monitoring; load test passed

Sprint 1 goal: baseline the requirements and architecture; enforce Jira issue keys on branches and commits; deliver authentication and role-based access control; and complete the priority-scheduling design for review.

Item	Owner	Estimate
`CONCORD-2`, `3`: authentication endpoints hardening and tests	Pallav	5 pts
`CONCORD-24`: review of the role guard across all operations	Sanchit	3 pts
`CONCORD-7`: CRDT integration into the backend	Pallav	5 pts
`CONCORD-36`: design review of the priority score and sorted-set scheduler	Aryamann	3 pts
CRDT convergence test harness (NFR-P1, NFR-D3)	Tattwa	3 pts
Jira and GitHub integration: issue keys on branches, instructor and TA access	Aamir	2 pts

A sprint review and retrospective close each sprint; unfinished items are carried into the next sprint.

12 AI Engineering Log

12.1 Method

LLMs were used only to produce and check the requirements and design artifacts in this document, not to write code. The process follows a Human and LLM requirements engineering workflow: the team supplies stakeholder intent, domain knowledge and constraints; the LLM probes, drafts and transforms; the LLM is then used again to trace, classify and cross-reference; a separate verification pass attacks the output for ambiguity, gaps and contradictions; and the team accepts, rejects, prioritizes and baselines. As in SpecGen, *generation proposes and verification disposes*: no LLM output entered the document without a checking pass and a team decision.

Every prompt uses a structured form: **Context**, **Role**, **Task**, **Constraints** and **Expected Output**. As an "engineering prompt", each one also demands identifiers, rationale, assumptions, acceptance criteria, conflicts and unknowns, so that the output can be checked rather than merely read. For each interaction the log records the prompt, the LLM output, the team's evaluation, the errors or hallucinations found, the changes made, and the final accepted artifact with the reason.

Tools: Antigravity (Claude Sonnet 4.6) for drafting; Claude Code for verification and consistency checking.

AI-01: Stakeholder Elicitation	
RE activity / tool	Elicitation (Probe) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** the approved Deliverable 0 proposal for Concord (collaborative editing, sandboxed execution, shared debugging, AI assistant, three roles). **Role:** requirements analyst. **Task:** interview three personas (a student, an instructor or TA, a course operator), identify each persona's needs, and list what is still unknown. **Constraints:** do not propose features outside the proposal; mark every assumption. **Expected output:** a stakeholder table, candidate user requirements with IDs and rationale, and a list of open questions.
LLM output	Stakeholder table; 14 candidate user requirements; open questions on role granularity, persistence of unsaved work, and behaviour when many users run code at once.
Evaluation	Stakeholder groups matched the proposal. The open questions were useful: they exposed that the three-role model could not express debugging rights, and that persistence was never stated.
Errors and rejections	Several candidates were features, not needs (for example a named UI panel); they were rewritten as needs. Candidates on grading and plagiarism detection were rejected as outside scope.
Changes made	A Debugger role was added between Editor and Owner; durable persistence of work was added; candidates were merged to nine.
Final artifact and reason	Section 1.1 and UR-01 to UR-09. Accepted with changes because the needs were traceable to real stakeholder goals once reworded.

AI-02: User Requirements in Natural Language	
RE activity / tool	Specification (Transform) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** the reviewed candidates from AI-01. **Role:** requirements engineer. **Task:** rewrite each as one numbered "shall" statement in natural language, one requirement per sentence. **Constraints:** no implementation terms; readable by students and instructors. **Expected output:** UR-ID, statement, rationale, source persona.
LLM output	Nine "shall" user requirements with rationale.
Evaluation	Most statements were atomic and understandable. Some used unmeasurable words ("instant", "fast", "secure").
Errors and rejections	"Instant propagation" and "fast execution" were ambiguous and not verifiable; they were kept as user wording but each was linked to a measurable NFR instead.
Changes made	Wording tightened; each vague adjective was traced to an NFR (NFR-P1, NFR-P2).
Final artifact and reason	Section 1.2. Accepted with changes: user requirements stay in user language, with measurability moved to the NFRs.

AI-03: Structured Specifications	
RE activity / tool	Specification (Draft) / Antigravity (Claude Sonnet 4.6)

Prompt	**Context:** UR-03, UR-04, UR-06 and UR-08. **Role:** systems analyst. **Task:** write structured specifications for four core transactions: processing a real-time edit, dispatching a code run, stepping a debug session, applying an AI-proposed edit. **Expected output:** one table per transaction with the fields Function, Description, Inputs, Outputs, Operations, Pre-conditions, Post-conditions and Side effects, with every failure path in Side effects.
LLM output	Four structured specifications in the standard template.
Evaluation	Operations and pre-conditions were consistent with the user requirements. Post-conditions were missing in two tables, and the failure paths were incomplete.
Errors and rejections	The AI-edit specification introduced a separate approval record and approval step that duplicated the normal edit path; rejected, because two write paths into a shared document create a consistency risk. A side effect claiming automatic rollback on queue failure was unsupported and was replaced by an explicit defect note.
Changes made	Post-conditions added; the AI edit is specified as an ordinary edit subject to the same role check; SPEC-05 was added later from AI-08.
Final artifact and reason	SPEC-01 to SPEC-05. Accepted with changes; the single write path keeps the specification consistent.

AI-04: Actors and Use-Case Model	
RE activity / tool	Specification (Graphical notation) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** UR-01 to UR-10 and the four-level role model. **Role:** UML modeller. **Task:** identify actors and use cases and their include and extend relationships. **Constraints:** follow standard UML use-case notation; actors must be outside the system boundary. **Expected output:** actor list with responsibilities, use-case list with IDs, relationship list, and diagram source.
LLM output	Actor taxonomy, 18 use cases, include and extend relations, diagram source.
Evaluation	Role inheritance was modelled correctly. Two "actors" were internal background processes (periodic saving and failure recovery), which lie inside the system boundary.
Errors and rejections	The internal processes were rejected as actors; one include relation was reversed and was corrected.
Changes made	Kept human actors plus two external system actors (execution worker and AI model provider).
Final artifact and reason	Section 2.1 and the use-case diagram. Accepted with changes because the corrected model respects the system boundary.

AI-05: User Stories	
RE activity / tool	Specification (Transform) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** the use cases from AI-04. **Role:** agile analyst. **Task:** write narrative user stories for joining a project, co-editing, running code, contended execution, debugging, snapshots and the AI assistant. **Constraints:** every story must include at least one alternative or failure path and cite its related requirements. **Expected output:** US-ID, narrative, related FR and NFR IDs.
LLM output	Seven narrative stories.
Evaluation	Stories were readable and scenario-based. One story contradicted itself (a user both editing and holding read-only access), and two stories had no failure path.
Errors and rejections	The contradictory story was rejected and rewritten with a separate read-only user; stories that invented behaviour not present in any requirement were trimmed.
Changes made	Added failure paths (permission refusal, time limit, memory limit); linked each story to its FR and NFR IDs.
Final artifact and reason	US-01 to US-07. Accepted with changes; each story is now consistent with the requirements it cites.

AI-06: Functional Requirements, Including AI-Agent Behaviour	
RE activity / tool	Specification (Draft) / Antigravity (Claude Sonnet 4.6)

Prompt	**Context:** UR-01 to UR-10, SPEC-01 to SPEC-05, US-01 to US-07. **Role:** requirements engineer. **Task:** derive functional requirements; for the AI assistant, state separately what it may do autonomously, what needs human approval, and what it must never do. **Constraints:** one testable behaviour per requirement; mark Mandatory or Optional. **Expected output:** FR-ID, statement, priority, source UR, acceptance criterion.
LLM output	Functional requirement list with priorities and acceptance criteria.
Evaluation	Coverage of the user requirements was complete. Several items bundled two behaviours; two stated acceptance criteria that could not be tested.
Errors and rejections	A requirement allowing the assistant to apply "trivial" fixes without approval was rejected, because it violates the human-approval boundary (NFR-EXT-ETH1). Bundled requirements were split.
Changes made	Requirements split and renumbered; AI boundaries written explicitly into FR-20 to FR-22.
Final artifact and reason	Section 3.1, FR-01 to FR-26. Accepted with changes.

AI-07: Non-Functional Requirements and Metrics

RE activity / tool	Specification (Classify) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** the functional requirements and the deployment constraints (free-tier cloud, laboratory use). **Role:** quality engineer. **Task:** classify non-functional requirements into the standard taxonomy (product: usability, efficiency, dependability, security; organizational: environmental, operational, development; external: regulatory, ethical, legislative) and give each quality property a measurable metric. **Expected output:** NFR-ID, category, statement, target value, verification method.
LLM output	Classified NFR list and a metrics table covering speed, size, ease of use, reliability, robustness and portability.
Evaluation	The classification followed the standard taxonomy. Some targets were chosen without justification and some had no verification method.
Errors and rejections	A target of sub-50 ms propagation was rejected as unrealistic for the deployment budget; a legislative requirement citing a specific data-protection law was rejected as unsupported by any stakeholder need.
Changes made	Targets adjusted for realism; a verification method attached to each metric.
Final artifact and reason	Sections 3.2 and 3.3. Accepted with changes.

AI-08: Contended Resource and Mathematical Specification

RE activity / tool	Specification (Draft) / Claude Code
Prompt	**Context:** the Deliverable 0 review asked for a finite, exclusively allocated resource and a non-trivial, testable calculation rule. **Role:** requirements engineer. **Task:** identify such a resource in the execution workflow and specify its allocation rule as a mathematical specification. **Constraints:** the rule must be deterministic, testable and fair. **Expected output:** the resource definition, the formula with each term defined, the correctness property to test, and the requirements it creates.
LLM output	Bounded pool of execution slots; a priority score based on role; correctness property "no slot is assigned twice".
Evaluation	The resource and the correctness property were accepted. The role-only formula was checked against the fairness constraint.
Errors and rejections	**Error found:** a role-only score lets higher-role users starve lower-role users indefinitely, which violates the fairness constraint. Rejected as drafted.
Changes made	An ageing wait-time term and a compilation-cost term were added; UR-10, FR-14, FR-15, NFR-D5 and SPEC-05 were created.
Final artifact and reason	Section 5.4, UR-10, FR-14, FR-15, NFR-D5, SPEC-05, ADR-007. Accepted only after the fairness fix.

AI-09: Attack the Output (Requirements Verification)	
RE activity / tool	Verification (Find contradictions) / Claude Code
Prompt	**Context:** the complete draft of Sections 1 to 3. **Role:** independent reviewer. **Task:** find at least five ambiguities, missing requirements, unverifiable statements or contradictions. **Constraints:** cite the requirement ID for each issue and classify it as ambiguity, gap, conflict or unverifiable. **Expected output:** an issue list with a proposed correction for each.
LLM output	Issue list covering ambiguity, gaps, conflicts and unverifiable statements.
Evaluation	The team confirmed these issues: (1) conflict between sending live updates to Viewers and Viewers being read-only; (2) no requirement for a worker failing mid-run; (3) inconsistent time limits between a user story and the NFRs; (4) metrics with no verification method; (5) no behaviour specified under contended execution capacity; (6) an overstated claim that output is streamed while a program runs.
Errors and rejections	Two reported "conflicts" were false positives caused by the reviewer misreading role inheritance; rejected.
Changes made	Distribution separated from authorization; FR-16 added; limits unified; verification methods added; UR-10 linked to AI-08; FR-12 reworded to broadcast results on completion.
Final artifact and reason	Section 3.4 (validation checks). Accepted; this pass closes the generation and verification loop.

AI-10: Traceability Matrix	
RE activity / tool	Verification (Trace, Cross-reference) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** UR, FR, NFR, UC, SPEC and US identifiers and the Jira backlog. **Role:** requirements manager. **Task:** build forward and backward traceability from user requirements to functional requirements, use cases, specifications, Jira issues and planned tests. **Constraints:** use only identifiers that exist in the inputs; flag any requirement with no link. **Expected output:** three matrices and a list of orphan items.
LLM output	Three traceability matrices.
Evaluation	Forward coverage was complete after corrections.
Errors and rejections	**Hallucination found:** several test-suite names and two Jira keys did not exist in the inputs. They were removed and replaced by planned test categories and real issue keys.
Changes made	Every UR checked by hand to map to at least one FR and one use case.
Final artifact and reason	Section 4. Accepted with changes after manual cross-checking.

AI-11: Architecture Description and Decision Records	
RE activity / tool	Specification (Draft) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** the baselined requirements and NFRs. **Role:** software architect. **Task:** describe the system architecture (context, containers, components) and write decision records for concurrent editing, job dispatch, persistence of live documents, isolation of untrusted code, authentication, and choice of AI provider. **Constraints:** at least two alternatives per decision, each with a trade-off; relate every decision to the NFRs it serves. **Expected output:** C4 diagram sources and ADRs (context, decision, alternatives, rationale, trade-off).
LLM output	C4 diagram sources and ADR-001 to ADR-006.
Evaluation	Decisions were consistent with the NFRs (latency, isolation, cost).
Errors and rejections	**Rejected:** listing the second AI provider only as a discarded alternative; the team kept two providers behind one interface to avoid a single-provider dependency.
Changes made	ADR-006 rewritten; ADR-007 and ADR-008 added by the team.
Final artifact and reason	Sections 5 and 8. Accepted with changes.

AI-12: Design Patterns and Sequence Diagrams

RE activity / tool	Specification (Draft, Classify) / Antigravity (Claude Sonnet 4.6)
Prompt	**Context:** the architecture from AI-11 and catalogues of architectural and AI-specific patterns. **Role:** software designer. **Task:** map patterns to components with a justification each, and draw UML sequence diagrams for four core scenarios. **Constraints:** use standard UML notation (lifelines, `alt` and `loop` frames); name a pattern only if the design contains its defining structure. **Expected output:** pattern table and diagram sources.
LLM output	Pattern tables and four sequence diagrams.
Evaluation	Most mappings were justified; diagrams followed standard UML notation after redrawing.
Errors and rejections	**Rejected:** "Chain of Responsibility" for authorization (the design has one central check, not a chain of handlers) and "Repository" for data access (there is no repository layer); RAG and a multi-agent architecture rejected as unjustified for this scope.
Changes made	Labels corrected; SD-2 for priority-ordered slot allocation added by the team.
Final artifact and reason	Sections 6 and 7. Accepted with changes.

Human-in-the-loop statement: the LLMs were used as collaborators inside a controlled requirements engineering workflow, not as authorities on requirements. Deciding what stakeholders need, the priorities, trade-offs and feasibility, and final acceptance were done by the team, and every accepted artifact passed a verification step before it was baselined.